

第 20 课：路由表从哪里来——转发平面、控制平面与路由收敛

预计时间：1.75 ~ 2.5 小时

场景：适合课堂或手机阅读，不需要电脑，不要求先完成 Lab 6

前置：第 6 课最长前缀匹配、第 7 ~ 9 课 Ethernet/ARP、第 17 课排队

本课从一个被我们一直跳过的问题开始

以前的模型里，我们总是直接写：

```
10.1.2.0/24 → 下一跳 R-team → 接口 2
```

然后让路由器执行最长前缀匹配。

这解释了：

路由表已经存在时，一份数据报怎样选择出口？

但没有解释：

这条路由项最初是谁放进去的？链路断开以后，又是谁把它换掉？

如果路由器永远使用启动时写死的规则，一条链路断开后，它仍然会把数据报送向坏掉的出口。ARP、Ethernet 和重传都救不了这个错误，因为路由器从一开始就选错了方向。

本课要把“查表”和“造表”分开。

学习目标

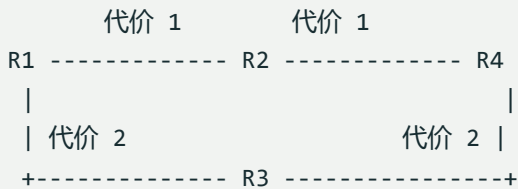
学完后，你应该能够：

1. 区分转发与路由；
2. 区分数据平面与控制平面；
3. 说明直连路由、静态路由和动态路由分别从哪里来；
4. 用自己的话比较距离向量与链路状态；

5. 解释 OSPF 与 BGP 为什么不解决同一个范围的问题；
6. 推演一次链路故障为什么会造成短暂丢包；
7. 区分 RIB、FIB、ARP 缓存和交换机 MAC 表；
8. 说明“路由协议选路”为什么不会改写数据报的最终目标 IP。

1. 已有模型遇到什么失败

考虑四台路由器：



R1 到 R4 有两条路：

R1 → R2 → R4，总代价 2
 R1 → R3 → R4，总代价 4

稳定时，R1 的规则可能是：

目标前缀：R4 后面的网络
 下一跳：R2
 出口接口：通往 R2 的接口

现在 R2-R4 链路断开。

R1 并不会凭空立刻知道这件事。如果它仍使用旧规则：

R1 → R2 → 断链处丢失

此时备用路径明明存在，数据报仍可能暂时丢失。

所以网络还需要一套机制：

发现拓扑变化
 → 把变化告诉需要知道的路由器
 → 重新计算可用路径
 → 更新实际转发表

这就是“路由”问题，而不仅是“转发”问题。

2. 转发和路由不是同一个动作

转发 forwarding

转发处理一份正在到来的数据报：

```
读取目标 IP
→ 查当前转发表
→ 得到出口接口和下一跳
→ TTL 减一
→ 交给出口网络接口
```

它的特点是：

- 每份数据报都要执行；
- 必须非常快；
- 使用已经准备好的规则；
- 不负责研究整个网络为什么应该走这条路。

路由 routing

路由解决的是规则怎样产生：

```
有哪些目标网络？
有哪些相邻路由器？
链路是否还活着？
多条路径中选择哪一条？
拓扑变化后什么时候改表？
```

它不需要为每份普通数据报重新运行一次完整算法，而是在拓扑或策略发生变化时更新规则。

一句话区分：

转发拿着现有规则处理这一份包；路由产生和更新未来要使用的规则。

3. 数据平面与控制平面

这两个词描述的是同一台路由器里的两类职责。

数据平面 data plane

数据平面负责高速处理普通数据报：

数据报进入

- 最长前缀匹配
- 选择下一跳和出口
- 排队并发送

第 4、6、7 课以及将来的路由器 Lab，主要实现的就是这部分。

控制平面 control plane

控制平面维护路由知识：

接收配置或路由协议消息

- 保存候选路径
- 选择最佳路径
- 把结果安装给数据平面

二者的关系不是“上层协议与下层协议”，而是**同一网络设备中的两类工作**：

控制平面：决定规则

↓ 安装

数据平面：高速使用规则

为什么必须分开

假设每到一份游戏数据报，路由器都重新收集整张互联网拓扑并运行一次寻路算法，显然来不及。

所以常见设计是：

1. 控制平面相对较慢地计算；
2. 把结果整理成适合快速查找的表；
3. 数据平面对大量数据报重复使用结果。

这和你写游戏时“偶尔重建导航网格，角色每帧查询已生成结构”有一点相似。但这只是计算频率的类比；真实路由协议还有分布式通信、策略和故障处理。

4. 最简单的路由项从哪里来

并非所有路由都需要动态协议。

4.1 直连路由

给接口配置：

接口 IP: 192.168.1.10
前缀长度: /24

设备就能推导出:

192.168.1.0/24 与这个接口直连

因此发往 192.168.1.77 时, 下一跳就是 192.168.1.77 本身。这正是 Lab 6 的第一步。

4.2 静态路由

管理员可以明确配置:

10.20.0.0/16 → 下一跳 192.168.1.1

优点:

- 行为清楚;
- 不需要运行动态路由协议;
- 小型、稳定网络中很好用。

缺点:

- 网络变大后维护量很大;
- 链路变化时不会自动理解新拓扑;
- 配置错误可能让整个前缀不可达。

4.3 默认路由

默认路由是:

0.0.0.0/0 → 默认网关

它能匹配所有 IPv4 地址, 但最长前缀匹配会让更具体的规则优先。

家庭电脑通常不需要知道整个互联网的路由。它只需知道:

直连网段 → 直接交付
其他目标 → 默认网关

5. 网络扩大后: 不能全靠人工维护

如果一个组织有数百台路由器和许多冗余链路, 人工在每台路由器上同步维护全部路径会遇到:

- 新网络加入时要修改许多设备；
- 链路断开时静态规则可能继续指向故障处；
- 多条路径的代价变化很难人工持续比较；
- 不同人的配置可能互相矛盾。

于是路由器需要交换“我知道什么”，让路由表随网络变化。

但这里有一个困难：

每台路由器只直接连接少数邻居，它没有一个天然存在的中央全知地图。

分布式路由算法就是在这个限制下工作。

6. 先看几个朴素方案

方案一：不知道就泛洪

收到一份数据报后，把副本发向除入口外的所有方向。

优点是不用先知道路径；缺点是副本会迅速膨胀，还要额外处理环路和重复。这适合帮助发现信息，不适合作为普通互联网数据报的长期转发方式。

不要把它和 Ethernet 交换机的未知单播泛洪混成一件事：

- 交换机泛洪局限在当前二层广播域；
- 这里讨论的是路由层面对多跳拓扑的一种朴素设想。

方案二：来源主机写完整路径

发送者把：

R1 → R2 → R4

整条路径放进报文，沿途设备照着走。这叫来源路由的一类思路。

问题是终端必须掌握拓扑和变化，而且完整路径会增加报文状态。现实协议中确实存在来源路由或显式路径机制，但普通 IP 转发主要使用逐跳查表：每台路由器只决定下一跳。

方案三：路由器分布式计算

让相邻路由器交换信息，各自逐渐形成可用路由。这是本课重点。

常见基础思路有两类：

距离向量：告诉邻居“我到各目标多远”
 链路状态：告诉大家“我有哪些链路及其代价”

7. 距离向量：我不必看见整张地图

假设 R1 的邻居只有 R2 和 R3。R1 想知道到目标 D 的最低代价。

邻居告诉它：

R2 说：我到 D 的代价是 1
 R3 说：我到 D 的代价是 2

R1 自己知道：

R1→R2 代价 1
 R1→R3 代价 2

于是比较：

经 R2: $1 + 1 = 2$
 经 R3: $2 + 2 = 4$

选择 R2。

形式化一点：

我到目标的估计
 $= \min(\text{到某个邻居的代价} + \text{邻居声称的目标代价})$

这对应 Bellman–Ford / 距离向量的核心递推思想。

它的重要特点：

- 路由器主要和邻居交换距离信息；
- 不要求每台路由器保存完整拓扑图；
- 信息逐跳传播；
- 坏消息传播可能较慢，错误估计可能造成临时环路。

“邻居说还有路”并不一定意味着邻居知道一条独立路径：邻居的估计可能又间接依赖你。这是距离向量中产生计数到无穷等问题的根源，后面会单独深入。

8. 链路状态：先同步地图，再各自运行 Dijkstra

链路状态采用另一种思路。

每台路由器描述自己的局部事实：

我是 R2
我连接 R1，代价 1
我连接 R4，代价 1

这些链路状态信息被可靠地传播到区域中的其他路由器。大家逐渐得到同一拓扑数据库，然后每台路由器都以自己为起点运行最短路径算法。

局部链路状态
→ 在区域内传播
→ 各路由器形成拓扑图
→ 各自运行 Dijkstra
→ 得到自己的下一跳

注意：大家拥有相似地图，不代表大家的转发表相同。

例如同一个目的网络：

R1 的下一跳可能是 R2
R3 的下一跳可能是 R4

因为最短路径树是以“当前这台路由器”为根计算的。

OSPF 是互联网内部常见的链路状态路由协议。它在一个自治系统内部传播链路状态，每台参与者从链路状态数据库计算最短路径树。[RFC 2328](#)

9. 路由收敛不是瞬间完成

链路断开后，通常要经历：

1. 相邻设备发现故障
2. 产生新的路由信息
3. 信息传播
4. 其他路由器重新计算
5. 新结果安装到转发表

从“真实拓扑已经变化”到“相关路由器都使用稳定新规则”的过程叫**收敛**。

在收敛期间，路由器掌握的信息可能不同步：

- 有的已经知道链路断开；

- 有的仍使用旧下一跳；
- 有的正在重新计算；
- 有的已经安装新路径。

因此可能暂时出现：

- 数据报丢失；
- 路径绕远；
- 短暂转发环路；
- RTT 和抖动突然变化；
- 数据报因不同路径而乱序到达。

这不是说路由协议“不可靠”，而是分布式系统不可能让所有设备在同一物理瞬间得知远处变化。

纸面轨迹：R2-R4 链路断开后的六个状态

一直比较三件事：真实拓扑、R1 控制平面的知识、R1 数据平面正在使用的 FIB。

步	发生的事件	控制平面目前知道	R1 当前 FIB	此刻发送结果
0	两条路径都正常	拓扑稳定	经 R2，代价 2	经 R2 到达
1	R2-R4 刚断开	R1 仍认为正常	仍经 R2	在断链处丢失
2	R2 检测并传播	更新正在路上	仍经 R2	仍可能丢失
3	R1 收到更新	已知链路失效	旧项等待替换	可能暂时丢失
4	R1 重新计算	新路径应经 R3	准备安装经 R3	处于切换边界
5	新 FIB 安装完成	知识与拓扑一致	经 R3，代价 4	经 R3 到达

旧路径：R1 -1-> R2 -1-> R4（后半段故障）

备用路径：R1 -2-> R3 -2-> R4

观察时一直区分两件事：

控制平面目前知道什么？
数据平面此刻仍在使用哪条 FIB 项？

10. RIB 与 FIB：候选知识不等于高速查表

设备实现中的具体名称会有差异，但常见区分是：

RIB: Routing Information Base

可以把 RIB 理解成控制平面的路由知识与候选结果：

直连路由
静态路由
OSPF 学到的路径
BGP 学到的路径

同一前缀可能存在多个候选来源，控制平面还要按策略、来源偏好和度量选择。

FIB: Forwarding Information Base

FIB 是提供给数据平面高速查询的转发结果，重点通常是：

目标前缀 → 下一跳/出口接口

第 6 课那张供最长前缀匹配使用的表，更接近我们对 FIB 的教学抽象。

可以先记成：

RIB：我知道和比较过哪些路
FIB：普通数据报现在实际按哪条规则走

这不是在要求你背设备厂商的内部实现，只是为了看清“协议刚学到变化”和“数据平面已经换表”可能不是同一时刻。

11. 最长前缀匹配仍然没有消失

路由协议负责产生候选路由，不会取代每包转发时的最长前缀匹配。

例如 FIB 中同时存在：

0.0.0.0/0	→ R-default
10.0.0.0/8	→ R-company
10.1.0.0/16	→ R-team
10.1.2.0/24	→ R-lab

目标 10.1.2.99 仍然选择 /24。

要把两个问题分开：

1. /24、/16 等规则是怎样学到和选入 FIB 的？——控制平面问题；
2. 一份目标为 10.1.2.99 的数据报匹配哪条已安装规则？——数据平面问题。

如果同一前缀有两条候选路径，控制平面会先选择，或者安装多条等价路径；之后数据平面才使用相应结果。OSPF 允许等价多路径，但本课程下一版 Lab 先只返回一个最佳下一跳。

12. 路由表、ARP 缓存和 MAC 表再次排成一条链

假设 R1 的 FIB 得出：

目标 203.0.113.0/24
→ 出口接口 2
→ 下一跳 IP 10.0.0.2

接下来仍然没有目标 MAC。

所以完整过程是：

目标 IP
↓ 查 FIB / 最长前缀匹配
出口接口 + 下一跳 IP
↓ 查 ARP 缓存，必要时 ARP
下一跳 MAC
↓ 制作 Ethernet 帧
交换机按 MAC 表选择端口

四张信息结构回答不同问题：

结构	典型键	典型结果	主要使用者
RIB	目标前缀与路由来源	候选路径、属性、度量	控制平面
FIB	目标 IP 的最长匹配前缀	下一跳、出口接口	路由器数据平面
ARP 缓存	当前链路下一跳 IP	下一跳 MAC	网络接口
交换机 MAC 表	目标 MAC	二层出口端口	交换机

不要把它们压缩成“一张万能地址表”。

13. 一个自治系统是什么

如果全世界所有路由器都参加同一个最短路径计算：

- 拓扑规模巨大；
- 每次变化传播范围过大；
- 不同运营者还拥有不同商业策略；

- 没有一个组织能替所有网络统一决定代价。

互联网因此划分为许多**自治系统**，简称 AS。可以先把它理解成：

由同一个网络管理主体控制，并对外采用共同路由策略的一组网络与路由器。

一个校园网络、云服务商或运营商可能运行一个或多个 AS；现实归属比这个简化描述更复杂。

于是路由问题分成两个尺度：

一个 AS 内部怎样到达各网络？
不同 AS 之间怎样交换可达性并执行策略？

14. OSPF：一个 AS 内部的链路状态路由

OSPF 属于内部网关协议的一种，典型用途是在一个 AS 内部：

1. 发现和维护邻接关系；
2. 传播链路状态信息；
3. 形成链路状态数据库；
4. 运行最短路径计算；
5. 产生到各目标网络的路由。

不要把“最短”自动理解成物理距离最短。OSPF 使用配置或计算出的链路代价，路径总代价较小者优先。

OSPF 的 Hello、链路状态更新、确认等细节很多，本课不要求背报文格式。现在需要掌握的是因果链：

局部链路事实
→ 区域内传播
→ 共同拓扑数据库
→ 每台路由器各自算树
→ 安装自己的下一跳

15. BGP：不同 AS 之间不仅比较“多远”

BGP 是 AS 之间交换路由可达性的核心协议。它更接近路径向量：一条路由会携带经过的 AS 路径等属性。

[RFC 4271](#)

例如两个出口都能到达某前缀：

路径甲：AS10 → AS20 → 目标
路径乙：AS10 → AS30 → AS40 → 目标

不能简单断言“AS 数更少的路径一定被选中”。运营者还会考虑：

- 客户、供应商与对等互联关系；
- 哪些路由允许对外发布；
- 哪个出口优先；
- 路由属性和本地策略。

所以：

OSPF 基础直觉：在一个管理域内按链路状态计算路径

BGP 基础直觉：在管理域之间交换可达路径并执行策略

BGP 不是为普通数据报增加一个新头部。它的控制消息帮助设备建立路由；之后普通 IP 数据报仍然逐跳查 FIB。

16. 为什么 BGP 不能简单换成全球 Dijkstra

即使计算能力足够，也还有管理和策略问题。

假设某运营商有两条可达路径：

- 一条技术上短，但经过昂贵的上游供应商；
- 一条多经过一个 AS，但来自互惠对等连接。

网络经营者可能根据策略选择第二条。这里的“最佳”不是单一全局距离函数决定的。

此外，组织通常不愿或不必向全世界公布内部每台路由器和每条链路。AS 把内部细节隐藏起来，只对外交换需要的可达性与路径属性。

因此互联网路由既是算法问题，也是分层管理和策略问题。

17. 控制平面消息自己怎样送达

这看起来有点递归：

路由协议要依靠网络发送消息，但网络又要靠路由协议产生路由。

启动时并非什么都没有。接口配置会先产生直连知识，管理员也可以配置必要的静态或默认路由。相邻路由器能借助直连链路建立邻接，再逐步交换更多路由信息。

可以理解为：

接口和直连关系提供最小起点

- 邻居之间能通信
- 路由协议交换更远处的信息
- 路由表逐步形成并收敛

18. 链路变化为什么会影响游戏体验

游戏应用通常看不到“R2—R4 刚刚断了”。它看到的是端到端结果：

- 某几份 UDP 快照丢失；
- TCP 暂时没有 ACK 前进并触发重传；
- RTT 突然升高；
- 切到备用路径后 RTT 基线改变；
- 不同数据报走不同路径造成乱序。

因此网络路径不是应用可以假定永远固定的管道。

应用层可采取的策略包括：

- 不把一个瞬时丢包立刻当作玩家断线；
- 对实时状态允许少量丢失并等待更新快照；
- 对关键结果使用确认或可靠通道；
- 让超时阈值和重连策略容忍短暂路径变化。

这里并不是要求游戏自己运行 OSPF/BGP，而是理解底层路径变化会以什么现象暴露给应用。

19. 与 Lab 6、Lab 7 的关系

Lab 6 中，主机接口只使用一个最小路由模型：

直连前缀
默认网关

它没有实现 OSPF 或 BGP。

后续 Lab 7 会实现：

- 一组已经给定的前缀规则
 - 最长前缀匹配
 - 出口接口与下一跳

这仍然主要属于数据平面。

把路由协议完整实现出来会远超一个教学 Lab 的规模。本课程会用小型图算法实验理解路由信息怎样产生，但不会假装几十行代码就是工业级 OSPF/BGP。

20. 常见混淆

混淆一：路由器每收到一个包就运行 Dijkstra

不是。控制平面在需要时计算并安装结果；数据平面重复查已准备的 FIB。

混淆二：OSPF 算出下一跳以后，数据报目标 IP 改成下一跳

不会。最终目标 IP 通常保持，下一跳用于当前一次链路交付。

混淆三：ARP 能发现远端网络的完整路径

ARP 只解决当前链路上一个下一跳 IP 对应哪个 MAC。它不计算多跳路径。

混淆四：BGP 总选 AS_PATH 最短的路径

不能这样概括。AS_PATH 是重要属性和防环依据之一，但选路还受其他属性和本地策略影响。

混淆五：链路恢复后所有路由器同时换表

不会。检测、传播、计算和安装都需要时间，收敛期间可能出现暂时不一致。

混淆六：路由表只有动态协议才能产生

直连配置和静态配置同样可以产生路由，动态协议只是另一类来源。

21. 本课最小模型

转发：处理这一份数据报，查 FIB 得到下一跳和出口

路由：产生和更新以后要使用的规则

控制平面：收集信息、选择路径、安装结果

数据平面：对大量数据报高速使用结果

直连路由：由接口地址和前缀产生

静态路由：由管理员配置

动态路由：由路由协议交换信息并计算

距离向量：向邻居传播“我到目标多远”

链路状态：传播局部链路事实，各自运行最短路径算法

OSPF：典型的 AS 内部链路状态路由协议

BGP：AS 之间交换可达路径并执行策略

RIB：控制平面的路由知识与候选

FIB：数据平面实际查找的转发结果

检查站

请遮住正文，用自己的话回答。可以画图，不要求背 RFC 细节。

1. 分别用一句话说明“转发”和“路由”。为什么普通数据报不适合每次都重新运行完整寻路算法？
2. 数据平面与控制平面各自负责什么？它们为什么不是网络分层中的上下两层？
3. 主机配置为 192.168.1.10/24，默认网关为 192.168.1.1。到 192.168.1.77 和 203.0.113.20 的路由知识分别主要来自什么？当前下一跳分别是谁？
4. 直连路由、静态路由和动态路由分别怎样产生？各举一个适合使用的场景。
5. 在 R1→R2→R4 最优、R1→R3→R4 备用的网络中，R2→R4 刚断开但 R1 尚未收到更新时，R1 为什么仍可能把数据报发给 R2？
6. 从链路断开到网络重新稳定，按顺序写出“检测、传播、重算、安装”四类动作。什么叫路由收敛？
7. 距离向量中的路由器通常向邻居报告什么？链路状态中的路由器向区域传播什么？两者谁通常需要形成完整拓扑图？
8. 为什么“OSPF 中各路由器拥有相似的链路状态数据库”不等于“它们的 FIB 完全相同”？
9. OSPF 与 BGP 主要解决的范围有什么不同？为什么 BGP 不能简单概括为“全球最短路径算法”？
10. 分别说明 RIB、FIB、ARP 缓存和交换机 MAC 表的键与结果大致是什么。
11. 一条路由由 OSPF 学到以后，一份普通数据报实际发送时仍要经过哪些步骤，才能得到 Ethernet 目标 MAC？
12. 链路切换期间，Unity 游戏可能观察到哪些现象？为什么一两份消息丢失不一定表示服务器宕机？

资料边界

本课结构与 Stanford CS144 的 Routing 单元一致：先问路由问题有哪些解法，再区分距离向量与链路状态。可参考 [CS144 Routing Lecture 1](#) 与 [Routing Lecture 2](#)。

协议细节以官方标准为准：OSPFv2 的链路状态数据库和最短路径树见 [RFC 2328](#)，BGP-4 的 AS 间路径交换见 [RFC 4271](#)。本课只建立机制主线，不要求记忆具体报文格式。

下一步

- 电脑旁：继续完成 Lesson 19 / Lab 6;
- 下一理论课：深入距离向量、环路与 count-to-infinity;
- 下一实现 Lab：最长前缀匹配与 IPv4 路由器。